Testing patterns

21C. Testing Patterns

Questions: 4 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Microsoft

---

Q 21.9 Does TDD work for frontend? When would you use red-green-refactor?

Difficulty: Mid **Asked at:** Google, Amazon, Atlassian

TDD (Test-Driven Development) means writing the test first (it fails — red), writing the minimum code to make it pass (green), then improving the code (refactor). Repeat.

Where TDD works well in frontend:

Utility functions and data transformations — when you know the input and expected output before writing the logic:

JAVASCRIPT Copy

```
// RED: Write the test first
test('formatRelativeTime returns "just now" for < 60 seconds', () => {
  expect(formatRelativeTime(30)).toBe('just now');
});

test('formatRelativeTime returns "5 minutes ago"', () => {
  expect(formatRelativeTime(300)).toBe('5 minutes ago');
});
```

```
});

// GREEN: Write the minimum code
function formatRelativeTime(seconds) {
  if (seconds < 60) return 'just now';
  if (seconds < 3600) return `${Math.floor(seconds / 60)} minutes ago`;
  return `${Math.floor(seconds / 3600)} hours ago`;
}

// REFACTOR: Handle edge cases, clean up
```

Custom hooks with defined behavior — when the hook's API is clear:

JAVASCRIPT Copy

```
// RED: Define expected behavior
test('useDebounce returns debounced value after delay', async () => {
  const { result } = renderHook(() => useDebounce('hello', 300));
  // ... test the timing behavior
});

// Then build the hook to pass the test
```

Where TDD doesn't work well in frontend:

- Visual/UI work — you don't know exactly what a component looks like until you see it
- Exploratory prototyping — requirements aren't clear enough for tests
- CSS and layout — automated tests can't evaluate visual design
- Components where the API is evolving rapidly — tests break with every refactor

The pragmatic approach most frontend teams use: Write the component first (you need to see it to know it's right), then write tests for the behavior you want to preserve. This is "test-after" development, not TDD, but it's realistic for UI work. Reserve true TDD for utility functions, hooks, and business logic.

Follow-up: "Do you practice TDD in your daily work?"

tests. For bug fixes, I always write a failing test that reproduces the bug before fixing it — this prevents regressions."

Q 21.10 When is snapshot testing useful and when is it noise?

Difficulty: Mid **Asked at:** Amazon, Flipkart, Atlassian

Snapshot testing serializes a component's rendered output (or any data structure) and saves it to a file. On subsequent test runs, it compares against the saved snapshot. If the output changed, the test fails.

JAVASCRIPT Copy

```
import { render } from '@testing-library/react';

test('ProductCard renders correctly', () => {
  const { container } = render(
    <ProductCard name="Running Shoes" price={2999} image="/shoes.jpg" />
  );
  expect(container).toMatchSnapshot();
});
```

When snapshots help:

- **Serialized data structures** — API response shapes, configuration objects, translation files. If the shape changes unexpectedly, the snapshot catches it.
- **Error messages** — verifying that error messages don't change accidentally.
- **Small, stable components** — a logo component, a footer, a static banner.

When snapshots are noise:

- **Large components** — a snapshot of a complex page is 500+ lines of HTML. Nobody reviews a 500-line diff. Developers blindly update the snapshot, defeating the

- **Frequently changing components** — every UI tweak triggers a snapshot update. The team starts updating snapshots reflexively without actually reviewing them.
- **Components with dynamic content** — dates, IDs, random values cause snapshots to fail even when nothing meaningful changed.

Inline snapshots are slightly better:

JAVASCRIPT Copy

```
test('formatPrice formats correctly', () => {
  expect(formatPrice(129900)).toMatchInlineSnapshot(`"₹1,299.00"`);
  expect(formatPrice(0)).toMatchInlineSnapshot(`"₹0.00"`);
});
```

Inline snapshots live in the test file, not a separate `.snap` file. They're visible in the code review, easier to reason about, and work well for small values.

The recommendation: Use behavioral tests (`expect(screen.getByText('...')).toBeVisible()`) as your primary testing approach. Use snapshots sparingly for data structures and serializable outputs where you want to detect any change. Never use snapshots as a substitute for thinking about what to test.

Follow-up: "Our team has 200 snapshot tests that nobody reviews. How would you fix this?"

Delete them. Replace with targeted behavioral tests that assert specific things ("the button is disabled when the form is invalid") rather than asserting the entire DOM. If a test can be updated by running `--updateSnapshot` without anyone looking at the diff, it's not providing value.

Q 21.11 How do you test async code — promises, timers, and race conditions?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart, Razorpay

JAVASCRIPT Copy

```javascript
async function fetchUserName(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) throw new Error('User not found');
  const data = await response.json();
  return data.name;
}

test('returns user name', async () => {
  const name = await fetchUserName('123');
  expect(name).toBe('Ankit');
});

test('throws on 404', async () => {
  server.use(
    http.get('/api/users/999', () => new HttpResponse(null, { status: 404
  );

  await expect(fetchUserName('999')).rejects.toThrow('User not found');
});
```

Testing timers (debounce, setTimeout, intervals):

JAVASCRIPT Copy

```javascript
import { vi, describe, it, expect, beforeEach, afterEach } from 'vitest';

beforeEach(() => vi.useFakeTimers());
afterEach(() => vi.useRealTimers());

test('debounce only calls function after delay', () => {
  const fn = vi.fn();
  const debounced = debounce(fn, 300);

  debounced('a');
  debounced('b');
  debounced('c');

  expect(fn).not.toHaveBeenCalled();
```

```
  expect(fn).toHaveBeenCalledTimes(1);
  expect(fn).toHaveBeenCalledWith('c');
});
```

Testing components with loading states:

JSX Copy

```jsx
test('shows loading then data', async () => {
  render(<UserProfile userId="123" />);

  expect(screen.getByText(/loading/i)).toBeInTheDocument();

  await waitFor(() => {
    expect(screen.getByText('Ankit')).toBeInTheDocument();
  });

  expect(screen.queryByText(/loading/i)).not.toBeInTheDocument();
});
```

Testing race conditions (outdated responses):

JAVASCRIPT Copy

```javascript
test('ignores stale API responses', async () => {
  const user = userEvent.setup();
  render(<SearchResults />);

  const input = screen.getByRole('textbox');

  await user.type(input, 'sho');
  await user.type(input, 'es');

  await waitFor(() => {
    expect(screen.getByText('Running Shoes')).toBeInTheDocument();
    expect(screen.queryByText('Shorts')).not.toBeInTheDocument();
  });
});
```

This test verifies that your component handles the case where an earlier API response returns after a later one. Components using `AbortController` or TanStack Query handle this correctly.

The `waitFor` pattern:

JAVASCRIPT Copy

```
await waitFor(() => {
  expect(screen.getByText('Loaded')).toBeInTheDocument();
}, { timeout: 3000 });

// findBy* is shorthand for waitFor + getBy
const element = await screen.findByText('Loaded');
```

Never use `setTimeout` or fixed delays in tests — they're slow and flaky. Always use `waitFor` or `findBy*` which retry with smart polling.

Q 21.12 How do you test custom hooks?

Difficulty: Mid **Asked at:** Amazon, Flipkart, Razorpay, Atlassian

Custom hooks can't be called outside of a React component. `renderHook` from React Testing Library creates a minimal wrapper component and returns the hook's return value.

Testing a simple state hook:

JAVASCRIPT Copy

```
import { renderHook, act, waitFor } from '@testing-library/react';

test('useToggle toggles between true and false', () => {
  const { result } = renderHook(() => useToggle(false));

  expect(result.current.value).toBe(false);

  act(() => {
    result.current.toggle();
```

```
  act(() => {
    result.current.toggle();
  });

  expect(result.current.value).toBe(false);
});
```

Testing a hook with dependencies (re-render with new props):

JAVASCRIPT Copy

```
test('useDebounce updates value after delay', () => {
  vi.useFakeTimers();

  const { result, rerender } = renderHook(
    ({ value, delay }) => useDebounce(value, delay),
    { initialProps: { value: 'hello', delay: 500 } }
  );

  expect(result.current).toBe('hello');

  rerender({ value: 'hello world', delay: 500 });

  expect(result.current).toBe('hello');

  act(() => vi.advanceTimersByTime(500));

  expect(result.current).toBe('hello world');

  vi.useRealTimers();
});
```

Testing an async hook (data fetching):

JAVASCRIPT Copy

```
test('useFetch returns data after loading', async () => {
  const { result } = renderHook(() => useFetch('/api/products'));
```

```
  await waitFor(() => {
    expect(result.current.loading).toBe(false);
  });

  expect(result.current.data).toHaveLength(3);
  expect(result.current.error).toBeNull();
});

test('useFetch handles errors', async () => {
  server.use(
    http.get('/api/products', () => new HttpResponse(null, { status: 500 }
  );

  const { result } = renderHook(() => useFetch('/api/products'));

  await waitFor(() => {
    expect(result.current.loading).toBe(false);
  });

  expect(result.current.error).toBeTruthy();
  expect(result.current.data).toBeNull();
});
```

Testing hooks that need providers (context, query client):

JAVASCRIPT Copy

```
function createWrapper() {
  const queryClient = new QueryClient({
    defaultOptions: { queries: { retry: false } },
  });

  return function Wrapper({ children }) {
    return (
      <QueryClientProvider client={queryClient}>
        {children}
      </QueryClientProvider>
    );
```

```
test('useProducts returns product list', async () => {
  const { result } = renderHook(() => useProducts('shoes'), {
    wrapper: createWrapper(),
  });

  await waitFor(() => {
    expect(result.current.isSuccess).toBe(true);
  });

  expect(result.current.data).toHaveLength(2);
});
```

When to test hooks directly vs through components:

Test hooks directly when: the hook is reused across many components (it's a shared utility), the hook has complex internal logic worth testing in isolation, or you want to test edge cases without rendering a full component.

Test hooks through components when: the hook is tightly coupled to one component, the behavior is best verified through user interaction, or the hook's output is purely visual (testing the rendered result makes more sense).

QUICK CHECK — no marks, just for you.

1. Why does testing a custom hook require renderHook instead of just calling the hook function directly in a test?

- ○ Calling the hook directly works, but renderHook is faster because it skips React's reconciler
- ○ renderHook is required because hooks return Promises that a plain function call would leave unresolved
- ○ A custom hook cannot be called outside a React component, so renderHook creates a minimal wrapper component and returns the hook's return value

renderHook is needed only for hooks that use useEffect; hooks built on useState can be called directly

Check answer

2. According to the lesson, what is the core reason snapshot tests of large components stop providing value?

- The diffs are so large that nobody reviews them, so developers blindly update the snapshot and the check becomes meaningless
- Snapshots of large components cannot be stored in a .snap file and must be inlined instead
- Snapshot serialization of a large component is too slow, so the suite times out before the assertion runs
- Large components render differently on every machine, so the snapshot can never match on CI

Check answer

3. A team asks whether they should apply strict red-green-refactor TDD to their UI components. Which position does this lesson take?

- Write snapshot tests before the component exists, so the first snapshot becomes the failing red step
- Skip tests for UI components entirely, since only utility functions can be meaningfully tested
- Apply red-green-refactor to every component, since a failing test is the only way to specify a component's markup up front
- Build the component first and then write tests for the behaviour worth preserving, reserving true TDD for utilities, hooks, and business logic

Previous
Advanced testing

Next
Linear data structures

LEARN MORE

ARTICLES & DOCS

Guiding principles (Testing Library)
testing-library.com

VIDEOS



React testing: pitfalls & best practices
YouTube



Testing React like a pro
YouTube



Best practices for testing React apps
YouTube

Hand-picked references for this lesson.